

CM 7

Apprentissage non supervisé

Info4.IA - Apprentissage automatique

Notes de cours

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

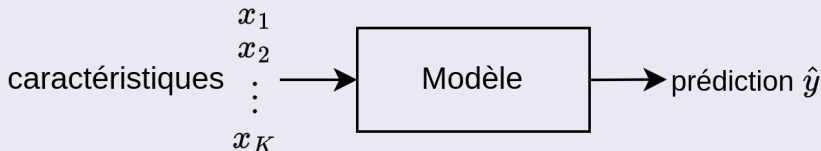
4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Rappel : Objectif

L'**objectif** de l'**apprentissage automatique** est d'élaborer un **modèle** qui :

- accepte en entrée des **caractéristiques** $x_1, x_2, \dots, x_K$
- fournit en sortie une **prédiction** $\hat{y}$



Apprentissage non supervisé et clustering

L'apprentissage est réalisé sur une série d'individus ayant les caractéristiques $x_1, x_2, \dots, x_K$.

Rappel : Vocabulaire

Si les caractéristiques $x_1, x_2, \dots, x_K$ sont accompagnées d'une **étiquette y connue**, on peut évaluer la qualité l'apprentissage par comparaison à cette étiquette. Il s'agit alors d'un **apprentissage supervisé**.

Dans le cas contraire, les "bonnes réponses" ne sont pas connues et ne permettent donc pas d'évaluer la qualité de l'apprentissage. Au lieu de cela, les inférences sont réalisées uniquement à partir de la distribution des caractéristiques. Il s'agit alors d'un **apprentissage non supervisé**.

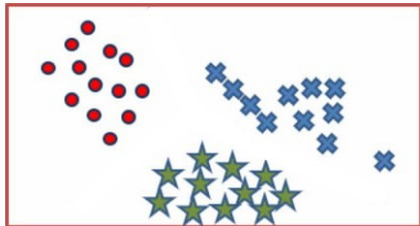
Apprentissage non supervisé et clustering

Exemple

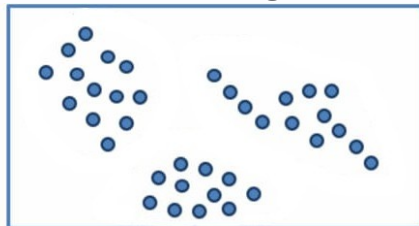
Exemple pour une tâche qui consisterait à regrouper des données :

- Caractéristiques : coordonnées (x_1, x_2) dans le plan.
- Prédiction : appartenance à l'une des 3 classes

Apprentissage supervisé
Classification



Apprentissage non supervisé
Clustering



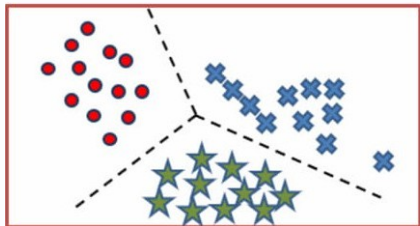
Apprentissage non supervisé et clustering

Exemple

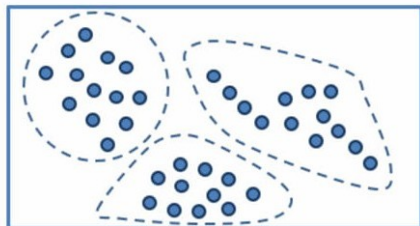
Exemple pour une tâche qui consisterait à regrouper des données :

- Caractéristiques : coordonnées (x_1, x_2) dans le plan.
- Prédiction : appartenance à l'une des 3 classes

Apprentissage supervisé
Classification



Apprentissage non supervisé
Clustering



Clustering

Le **clustering** (regroupement des données) consiste à associer entre elles des **données ayant des caractéristiques** $x_1, x_2, \dots, x_K$ **similaires**, de façon à former des ensembles ou clusters.

Ainsi, les éléments à l'intérieur d'un même cluster sont plus similaires les uns aux autres qu'aux éléments d'autres clusters.

Pour effectuer la mesure de similarité entre individus ou entre groupes, il est nécessaire de définir les notions de :

- distance entre individus.
- distance entre groupes.

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Distance entre individus

On considère des individus dont on connaît les K caractéristiques $x_1, \dots, x_K$.

On se place donc dans un espace à K dimensions, et un individu i est entièrement caractérisé par le point X_i de coordonnées $(x_{i1}, \dots, x_{iK})$ dans cet espace à K dimensions.

Étant donnés deux points X_i et X_j de coordonnées respectives $(x_{i1}, \dots, x_{iK})$ et $(x_{j1}, \dots, x_{jK})$, on souhaite alors définir la distance entre ces deux points X_i et X_j .

Homogénéité des caractéristiques

Pour que les calculs de distances soient pertinents, il faut s'assurer que les échelles des différentes caractéristiques sont homogènes. Une étape préalable de **centrage-réduction** des données selon chaque dimension peut s'avérer nécessaire.

Distance entre individus

Une première **formule générale** est celle de la distance de la norme p :

Distance de la norme p

Étant donnés deux points X_i et X_j de coordonnées respectives $(x_{i1}, \dots, x_{iK})$ et $(x_{j1}, \dots, x_{jK})$, on définit la **distance de la norme p** entre ces deux points par :

$$d_p(X_i, X_j) = \left(\sum_{k=1}^K |x_{ik} - x_{jk}|^p \right)^{\frac{1}{p}}$$

Distance entre individus

De cette formule générale on peut dériver différentes distances déjà connues.

Lorsque $p = 1$ on retrouve la distance de la norme 1, ou **distance de Manhattan** :

Distance de la norme 1

Étant donnés deux points X_i et X_j de coordonnées respectives $(x_{i1}, \dots, x_{iK})$ et $(x_{j1}, \dots, x_{jK})$, la **distance de Manhattan** entre ces deux points est :

$$d_1(X_i, X_j) = \sum_{k=1}^K |x_{ik} - x_{jk}|$$

Distance entre individus

Lorsque $p = 2$ on retrouve la distance de la norme 2, ou **distance euclidienne** :

Distance de la norme 2

Étant donnés deux points X_i et X_j de coordonnées respectives $(x_{i1}, \dots, x_{iK})$ et $(x_{j1}, \dots, x_{jK})$, la **distance euclidienne** entre ces deux points est :

$$d_2(X_i, X_j) = \left(\sum_{k=1}^K |x_{ik} - x_{jk}|^2 \right)^{\frac{1}{2}} = \sqrt{\sum_{k=1}^K (x_{ik} - x_{jk})^2}$$

Distance entre individus

Lorsque p tend vers $+\infty$, le terme $|x_{ik} - x_{jk}|$ prépondérant est celui qui a la valeur maximale, et c'est lui qui détermine la valeur de la distance.

On peut alors définir la distance de la norme ∞ , ou **distance de Tchebychev** :

Distance de la norme ∞

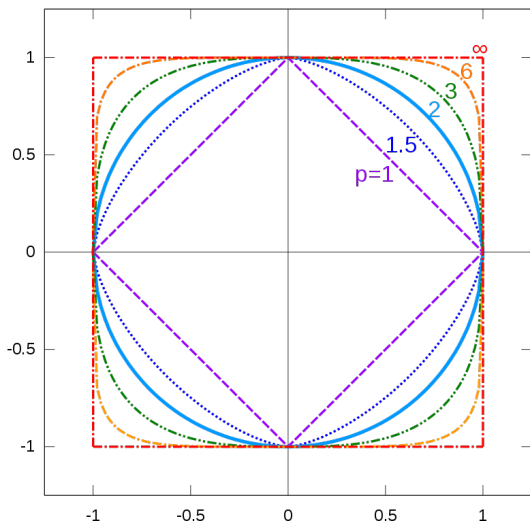
Étant donnés deux points X_i et X_j de coordonnées respectives $(x_{i1}, \dots, x_{iK})$ et $(x_{j1}, \dots, x_{jK})$, la **distance de Tchebychev** entre ces deux points est :

$$d_{\infty}(X_i, X_j) = \max_{1 \leq k \leq K} |x_{ik} - x_{jk}|$$

Distance entre individus

La figure ci-contre montre les ensembles des points à distance 1 de l'origine pour différentes valeurs de p , en dimension $K = 2$.

- Pour $p = 1$, l'ensemble forme un carré tourné en "diamant".
- Pour $p = 2$, l'ensemble forme un cercle de rayon 1.
- Pour $p = \infty$, l'ensemble forme un carré.



Distance entre individus

Autres mesures possibles :

- **Percent disagreement (ou distance de Hamming)**: cette mesure est particulièrement utile si les données ne sont pas numériques

$$d_H(X_i, X_j) = \frac{\text{Nombre de } x_{ik} \neq x_{jk}}{K}$$

- **Distance cosinus** : ce score est fréquemment utilisée en fouille de textes, et est caractérisé par la distance angulaire :

$$d_C(X_i, X_j) = 1 - \cos \theta_{X_i, X_j} = 1 - \frac{X_i \cdot X_j}{\|X_i\|_2 \|X_j\|_2}$$

Distance entre individus

Exemple & Implémentation

On donne deux points X_1 et X_2 de coordonnées respectives $(1, 2, 3, 4)$ et $(2, 2, 0, 4)$

1) Calculer $d_1(X_1, X_2)$, $d_2(X_1, X_2)$ et $d_\infty(X_1, X_2)$.

2) a) Écrire les fonctions Python `distance_1`, `distance_2` et `distance_inf` qui acceptent en paramètres deux tableaux Numpy `x1` et `x2` de même longueur et retournent respectivement les valeurs des distances de norme 1, 2 et ∞ entre `x1` et `x2`.

b) Retrouver les résultats du calcul précédent.

Distance entre individus

Exemple & Implémentation

On donne deux points X_1 et X_2 de coordonnées respectives $(1, 2, 3, 4)$ et $(2, 2, 0, 4)$

1) Calculer $d_1(X_1, X_2)$, $d_2(X_1, X_2)$ et $d_\infty(X_1, X_2)$.

2) a) Écrire les fonctions Python `distance_1`, `distance_2` et `distance_inf` qui acceptent en paramètres deux tableaux Numpy `x1` et `x2` de même longueur et retournent respectivement les valeurs des distances de norme 1, 2 et ∞ entre `x1` et `x2`.

b) Retrouver les résultats du calcul précédent.

$$1) d_1(X_1, X_2) = |1 - 2| + |2 - 2| + |3 - 0| + |4 - 4| = 4$$

$$d_2(X_1, X_2) = \sqrt{(1 - 2)^2 + (2 - 2)^2 + (3 - 0)^2 + (4 - 4)^2} = \sqrt{10} \approx 3,162$$

$$d_\infty(X_1, X_2) = \max\{|1 - 2|, |2 - 2|, |3 - 0|, |4 - 4|\} = 3$$

Distance entre individus

2)

```
1 def distance_1(X1,X2):  
2     return sum(abs(X1-X2))  
3  
4 def distance_2(X1,X2):  
5     return sum((X1-X2)**2)**(1/2)  
6  
7 def distance_inf(X1,X2):  
8     return max(abs(X1-X2))  
9  
10 X1 = np.array([1,2,3,4])  
11 X2 = np.array([2,2,0,4])  
12 print(distance_1(X1,X2))  
13 print(distance_2(X1,X2))  
14 print(distance_inf(X1,X2))
```

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Distance entre groupes

Un fois que l'on a décidé comment on calcule la distance entre deux points, on souhaite avoir une mesure agrégative de la distance entre deux ensembles de points A et B de cardinaux respectifs n_A et n_B . Les critères les plus fréquemment utilisés sont les suivants :

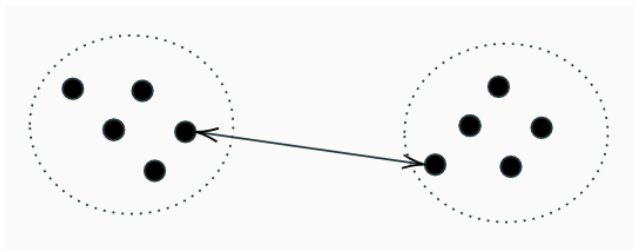
- Saut minimum.
- Saut maximal.
- Lien moyen.
- Distance entre centroïdes.
- Distance de Ward.

Distance entre groupes

Saut minimum

Le **saut minimum** entre A et B est défini comme la plus petite distance entre deux points de A et B :

$$D(A, B) = \min_{X_i \in A, X_j \in B} d(X_i, X_j)$$

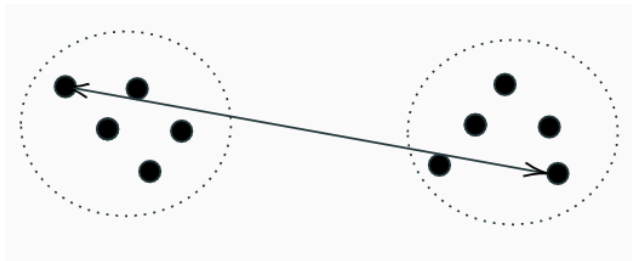


Distance entre groupes

Saut maximum

Le **saut maximum** entre A et B est défini comme la plus grande distance entre deux points de A et B :

$$D(A, B) = \max_{X_i \in A, X_j \in B} d(X_i, X_j)$$

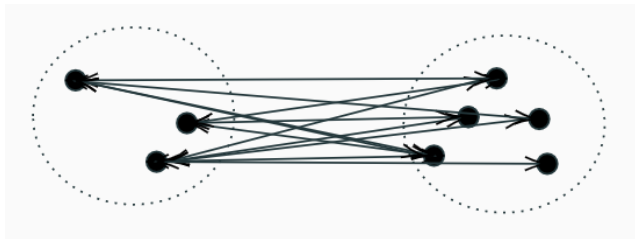


Distance entre groupes

Lien moyen

Le **lien moyen** entre A et B est défini comme la moyenne des distances entre des couples de points de A et B :

$$D(A, B) = \frac{1}{n_A \cdot n_B} \sum_{X_i \in A, X_j \in B} d(X_i, X_j)$$



Distance entre groupes

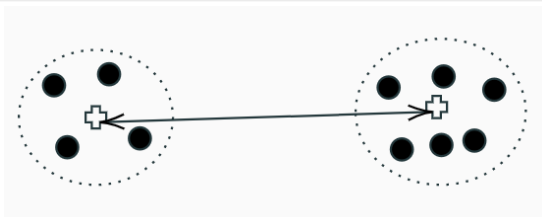
Distance entre centroïdes

Pour chaque ensemble de points A et B , on peut définir leurs centres de gravité (centroïdes) respectifs.

$$G_A = \frac{1}{n_A} \sum_{X \in A} X \qquad G_B = \frac{1}{n_B} \sum_{X \in B} X$$

La **distance entre centroïdes** entre A et B est définie comme la distance entre G_A et G_B :

$$D(A, B) = d(G_A, G_B)$$



Distance de Ward

La **distance de Ward** entre A et B est définie par :

$$D_W(A, B) = \sqrt{\frac{n_A \cdot n_B}{n_A + n_B}} d_2(G_A, G_B)$$

Remarque : Une propriété intéressante de cette distance est donnée en fin de cours (*Évaluation des solutions obtenues*).

Implémentation

On suppose que :

- les groupes de points sont représentés par des tableaux Numpy de dimension (n, K) où n est le nombre de points du groupe et K le nombre de dimensions.
- La distance entre deux points est la distance euclidienne.

Écrire les fonction `linkage_min`, `linkage_max`, `linkage_average`, `linkage_centroids` et `linkage_ward` qui acceptent en paramètres deux groupes de points A et B et retournent la distance entre groupes correspondante.

Distance entre groupes

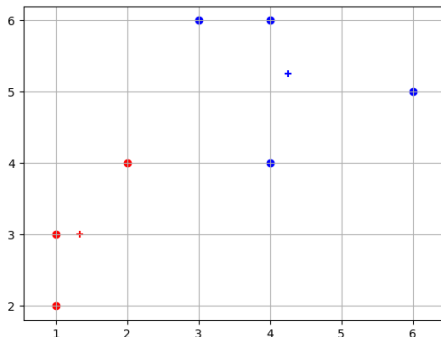
```
1 def linkage_min(A,B):
2     return min(min(distance_2(XA,XB)
3                 for XA in A) for XB in B)
4
5 def linkage_max(A,B):
6     return max(max(distance_2(XA,XB)
7                 for XA in A) for XB in B)
8
9 def linkage_average(A,B):
10    return sum(sum(distance_2(XA,XB)
11                 for XA in A) for XB in B)/len(A)/len(B)
12
13 def linkage_centroids(A,B):
14     GA = np.average(A,axis=0)
15     GB = np.average(B,axis=0)
16     return distance_2(GA,GB)
```

Distance entre groupes

```
1 def linkage_ward(A,B):
2     nA = len(A)
3     nB = len(B)
4     return (np.sqrt(len(A)*len(B)/(nA+nB))
5             *linkage_centroids(A,B))
6
7 # Exemple
8 A = np.array([[1,2],[1,3],[2,4]])
9 B = np.array([[3,6],[4,6],[6,5],[4,4]])
10 print('Min :',linkage_min(A,B))
11 print('Max :',linkage_max(A,B))
12 print('Average :',linkage_average(A,B))
13 print('Centroids :',linkage_centroids(A,B))
14 print('Ward :',linkage_ward(A,B))
```

Distance entre groupes

Min : 2.0
Max : 5.830951894845301
Average : 3.8743228569215553
Centroids : 3.6836726842167242
Ward : 4.823059688818371



- 1 Introduction
 - Apprentissage non supervisé et clustering
 - Distance entre individus
 - Distance entre groupes
- 2 Classification Ascendante Hiérarchique
 - Principe général
 - Exemple d'implémentation
 - Utilisation de `scipy`
- 3 K-means
 - Principe général
 - Exemple
- 4 Évaluation des solutions obtenues
 - WCSS et méthode du coude
 - Propriété de la distance de Ward

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

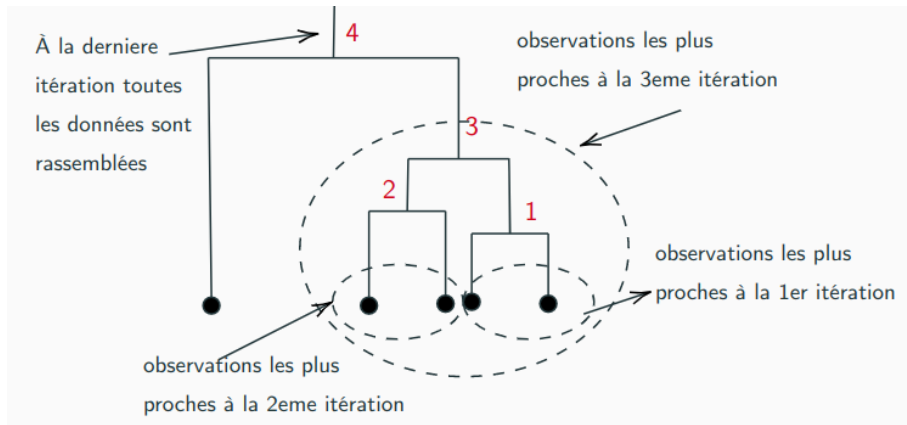
Principe général

Le principe de la **Classification Ascendante Hiérarchique** est de regrouper itérativement les individus :

- On commence par grouper les individus les plus proches.
- On regroupe au fur et à mesure les groupes les plus proches.

On construit alors progressivement un arbre, ou **dendrogramme**, qui regroupe finalement tous les individus en une seule classe, à la racine.

Principe général



1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Exemple d'implémentation

Implémentation

On suppose que :

- L'ensemble des groupes de points est représenté par une liste (native Python) de tableaux Numpy. (*En effet, on ne peut pas regrouper des tableaux Numpy de tailles différentes dans un même tableau Numpy*).
- Les noms des groupes sont données dans une liste séparée.
- La distance entre deux points est la distance euclidienne.
- La distance entre deux groupes est la distance entre centroïdes.

1) Écrire la fonction `nearest_clusters` qui accepte en paramètre une liste de clusters data et retourne les indices des clusters les plus proches.

Exemple d'implémentation

Implémentation (suite)

2) Écrire la fonction `agglomerate` qui accepte en paramètres :

- Une liste de clusters `data`
- Une liste de noms `labels`
- Deux indices `i` et `j`

et a pour effet de fusionner les données d'indices `i` et `j` dans `data` et `labels`.

3) Écrire la fonction `hierarchical_clustering` qui accepte en paramètres :

- La liste initiales de clusters (singletons) `data`
- La liste des noms des points initiaux `labels`

et implémente l'algorithme de classification Ascendante Hiérarchique.

Exemple d'implémentation

```
1 def nearest_clusters(data):
2     n = len(data)
3     i_min, j_min = 0, 1
4     for i in range(0, n):
5         for j in range(i+1, n):
6             if (linkage_centroids(data[i], data[j])
7                 < linkage_centroids(data[i_min], data[j_min])):
8                 i_min, j_min = i, j
9     return i_min, j_min
10
11 def agglomerate(data, labels, i, j):
12     data[i] = np.concatenate((data[i], data[j]))
13     data.pop(j)
14     labels[i] = labels[i] + labels[j]
15     labels.pop(j)
```

Exemple d'implémentation

```
1 def hierarchical_clustering(data, labels):
2     while len(data)>1:
3         i_min, j_min = nearest_clusters(data)
4         print('Fusion', labels[i_min], labels[j_min],
5               linkage_centroids(data[i_min], data[j_min]))
6         agglomerate(data, labels, i_min, j_min)
```

Exemple d'implémentation

```
1 data = [  
2     np.array([[160, 50]]),  
3     np.array([[170, 58]]),  
4     np.array([[155, 48]]),  
5     np.array([[165, 59]]),  
6     np.array([[180, 71]]),  
7     np.array([[175, 90]])  
8 ]  
9 labels = ['A','B','C','D','E','F']  
10 hierarchical_clustering(data,labels)
```

```
Fusion B D 5.0990195135927845  
Fusion A C 5.385164807134504  
Fusion AC BD 13.793114224133722  
Fusion E F 19.6468827043885  
Fusion ACBD EF 30.66859142510461
```

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Utilisation de scipy

Implémentation (suite)

4) Comparer avec la classification obtenue grâce à la librairie `scipy`.

Utilisation de scipy

Implémentation (suite)

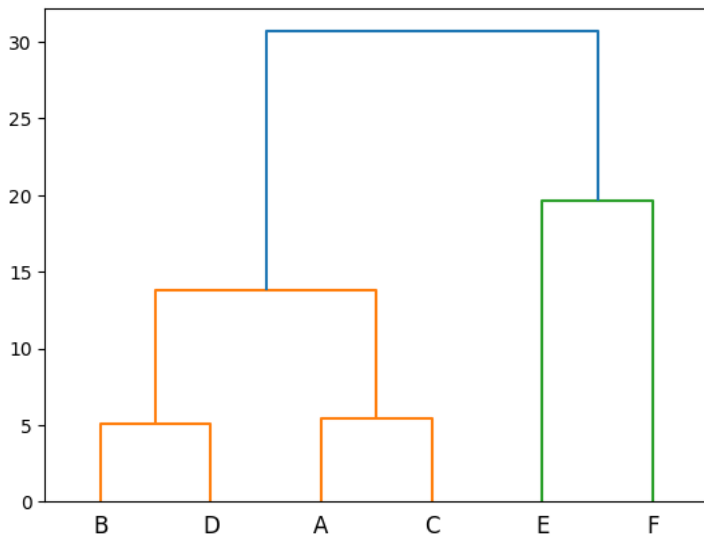
4) Comparer avec la classification obtenue grâce à la librairie scipy.

```
1 import matplotlib.pyplot as plt
2 from scipy.cluster.hierarchy import dendrogram,
   linkage
3
4 data = np.array(
5     [[160, 50],[170, 58],[155, 48],
6      [165, 59],[180, 71],[175, 90]])
7 centroid_linkage = linkage(data, metric='euclidean',
8                             method='centroid')
9 print(centroid_linkage)
10 dendrogram(centroid_linkage,
11             labels=['A', 'B', 'C', 'D', 'E', 'F'])
12 plt.show()
```

Utilisation de scipy

```
[[ 1.          3.          5.09901951  2.          ]
 [ 0.          2.          5.38516481  2.          ]
 [ 6.          7.         13.79311422  4.          ]
 [ 4.          5.         19.6468827   2.          ]
 [ 8.          9.         30.66859143  6.          ]
  ]]
```

Utilisation de scipy



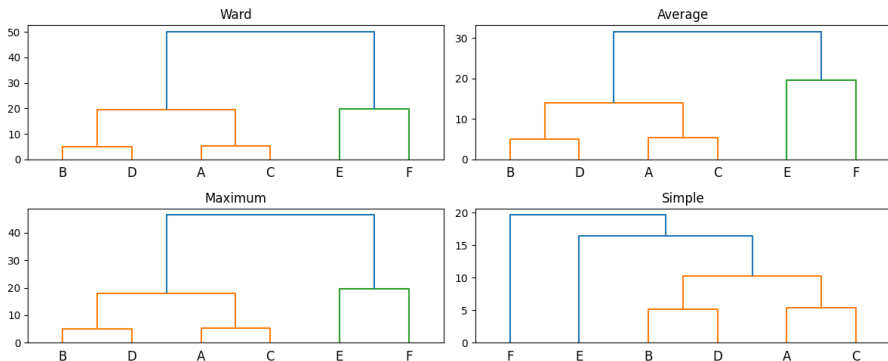
Utilisation de scipy

La fonction `linkage` de la librairie `scipy` permet de choisir :

- La distance entre points (*paramètre `metric=...`*) :
 - `'cityblock'` : Distance de Manhattan
 - `'euclidean'` : Distance Euclidienne
 - `'chebyshev'` : Distance de Tchebychev (norme ∞)
 - `'hamming'` : Percent disagreement (distance de Hamming)
 - `'cosine'` : Distance cosinus
- La distance entre groupes (*paramètre `method=...`*) :
 - `'single'` : saut minimum
 - `'complete'` : saut maximum
 - `'average'` : lien moyen
 - `'centroid'` : distance entre centroïdes
 - `'ward'` : distance de Ward (à un facteur $\sqrt{2}$ près)

Utilisation de scipy

Pour d'autres distances entre groupes, on obtient pour les données de l'exemple précédent :



1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

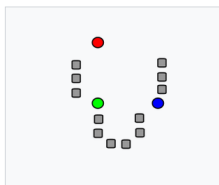
- Principe général
- Exemple

4 Évaluation des solutions obtenues

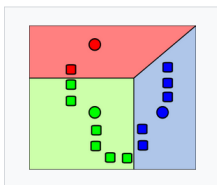
- WCSS et méthode du coude
- Propriété de la distance de Ward

Principe général

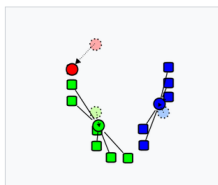
Le principe de l'**agrégation autour de centres mobiles** consiste à allouer directement les données aux groupes, puis de mettre à jour les groupes itérativement.



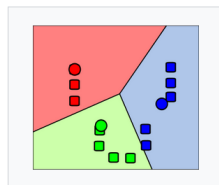
On initialise les centres des groupes, par tirage aléatoire par exemple.



Chaque individu est affecté au centre dont il est le plus proche.



Calcul des centroïdes des K classes ainsi constituées.



On répète l'opération sur les nouveaux centroïdes.

Principe général

Voici l'algorithme K-means proposé par Mac Queen (1967)

- **Initialisation** : Tirer au hasard, ou sélectionner pour des raisons extérieures à la méthode, K points dans l'espace des individus.
- **1 : Allouer** chaque individu au centroïde le plus proche au sens de la distance choisie. On obtient ainsi, à chaque étape, une classification en K classes, ou moins si, finalement, une des classes devient vide.
- **2 : Calculer** le barycentre de chaque classe. Il devient le nouveau centroïde si une classe s'est vidée, on peut éventuellement retirer aléatoirement un noyau complémentaire.
- **3 : Répéter 1 et 2**
- **Arrêt** lorsque les classes sont stabilisées.

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

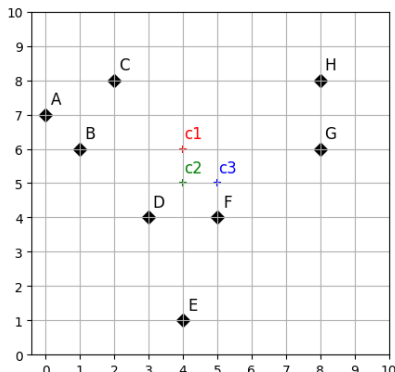
- WCSS et méthode du coude
- Propriété de la distance de Ward

Exemple

Exemple

On considère les points A, B, ..., H dont les coordonnées sont les suivantes :

A	B	C	D	E	F	G	H
(0,7)	(1,6)	(2,8)	(3,4)	(4,1)	(5,4)	(8,6)	(8,8)



Regrouper ces 8 points avec les paramètres suivants :

- distance euclidienne.
- $K = 3$ groupes.
- centroïdes initiaux
 - $c1(4,6)$
 - $c2(4,5)$
 - $c3(5,5)$

Exemple

On calcule la distance euclidienne entre chacun des centroides et les points

	A	B	C	D	E	F	G	H
c1	4.12	3.00	2.83	2.24	5.00	2.24	4.00	4.47
c2	4.47	3.16	3.61	1.41	4.00	1.41	4.12	5.00
c3	5.39	4.12	4.24	2.24	4.12	1.00	3.16	4.24

Exemple

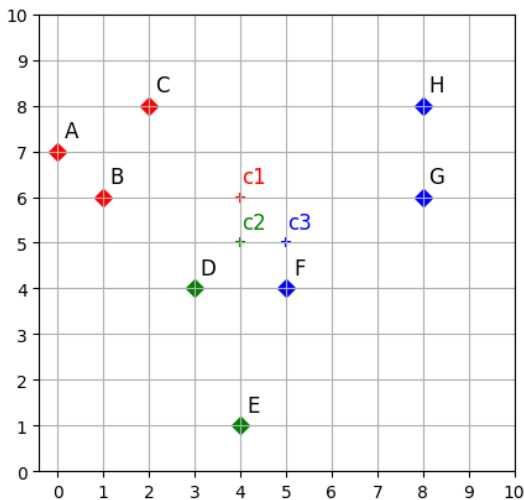
On calcule la distance euclidienne entre chacun des centroïdes et les points

	A	B	C	D	E	F	G	H
c1	4.12	3.00	2.83	2.24	5.00	2.24	4.00	4.47
c2	4.47	3.16	3.61	1.41	4.00	1.41	4.12	5.00
c3	5.39	4.12	4.24	2.24	4.12	1.00	3.16	4.24

On assigne les points à leur centroïde le plus proche :

- c1 : A, B, C
- c2 : D, E
- c3 : F, G, H

Example

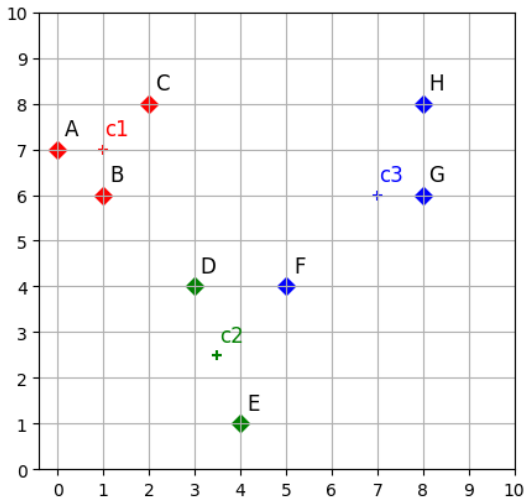


Exemple

Nouvelles coordonnées des 3 centroïdes :

- $c1 : (1,7)$
- $c2 : (3.5,2.5)$
- $c3 : (7,6)$

Exemple



Exemple

On calcule la distance euclidienne entre chacun des centroides et les points

	A	B	C	D	E	F	G	H
c1	1.00	1.00	1.41	3.61	6.71	5.00	7.07	7.07
c2	5.70	4.30	5.70	1.58	1.58	2.12	5.70	7.11
c3	7.07	6.00	5.39	4.47	5.83	2.83	1.00	2.24

Exemple

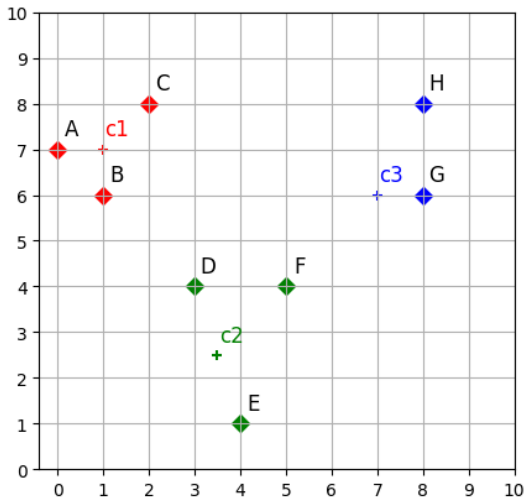
On calcule la distance euclidienne entre chacun des centroïdes et les points

	A	B	C	D	E	F	G	H
c1	1.00	1.00	1.41	3.61	6.71	5.00	7.07	7.07
c2	5.70	4.30	5.70	1.58	1.58	2.12	5.70	7.11
c3	7.07	6.00	5.39	4.47	5.83	2.83	1.00	2.24

On assigne les points à leur centroïde le plus proche :

- c1 : A, B, C
- c2 : D, E, F
- c3 : G, H

Example

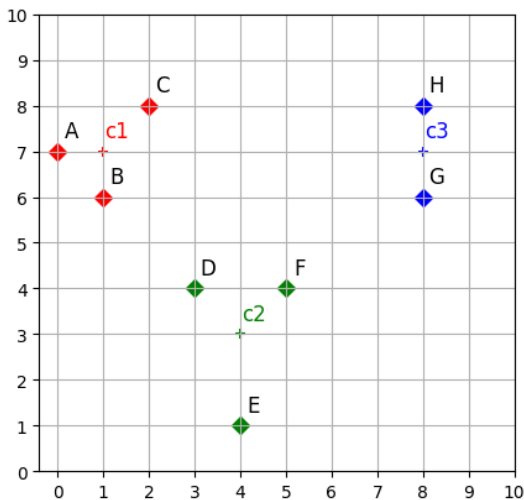


Exemple

Nouvelles coordonnées des 3 centroïdes :

- $c1 : (1,7)$
- $c2 : (4,3)$
- $c3 : (8,7)$

Exemple



Exemple

On calcule la distance euclidienne entre chacun des centroides et les points

	A	B	C	D	E	F	G	H
c1	1.00	1.00	1.41	3.61	6.71	5.00	7.07	7.07
c2	5.66	4.24	5.39	1.41	2.00	1.41	5.00	6.40
c3	8.00	7.07	6.08	5.83	7.21	4.24	1.00	1.00

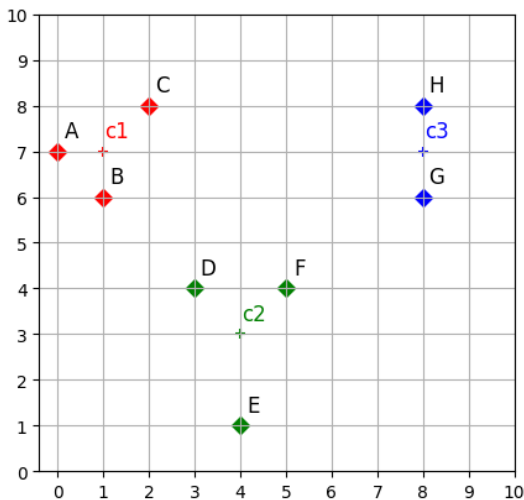
Exemple

On calcule la distance euclidienne entre chacun des centroides et les points

	A	B	C	D	E	F	G	H
c1	1.00	1.00	1.41	3.61	6.71	5.00	7.07	7.07
c2	5.66	4.24	5.39	1.41	2.00	1.41	5.00	6.40
c3	8.00	7.07	6.08	5.83	7.21	4.24	1.00	1.00

Pas de changement dans les affectations : l'algorithme est terminé

Example



1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Afin d'évaluer la qualité de la solution obtenue, on propose l'indicateur suivant :

Définition

Étant donné un ensemble de points X_i et un clustering caractérisé par les centroïdes $\mathcal{C}(X_i)$ associé à chaque point, on définit la **somme des carrés au sein des clusters** (*Within Cluster Sum of Squares*, *WCSS*) par :

$$WCSS = \sum_{X_i} d(X_i, \mathcal{C}(X_i))^2$$

WCSS et méthode du coude

Cet indicateur peut servir à déterminer le nombre K de clusters qui est le plus pertinent dans l'algorithme K-Means.

Exemple (suite)

Tracer la valeur de la WCSS selon la valeur du nombre K de clusters et en déduire le nombre de clusters le plus pertinent.

WCSS et méthode du coude

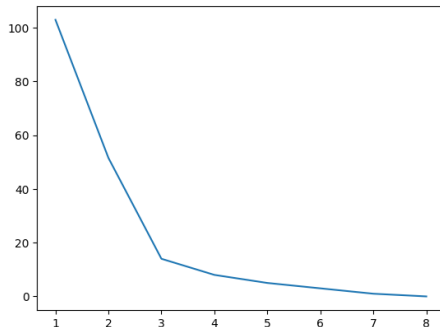
Cet indicateur peut servir à déterminer le nombre K de clusters qui est le plus pertinent dans l'algorithme K-Means.

Exemple (suite)

Tracer la valeur de la WCSS selon la valeur du nombre K de clusters et en déduire le nombre de clusters le plus pertinent.

```
1 points =  
    [(0,7), (1,6), (2,8), (3,4), (4,1), (5,4), (8,6), (8,8)]  
2 wcss = []  
3 for K in range(1,9):  
4     kmeans = KMeans(n_clusters=K, n_init=10)  
5     kmeans.fit(points)  
6     wcss.append(kmeans.inertia_)  
7 plt.plot(range(1,9), wcss)
```

WCSS et méthode du coude



Méthode du coude

Le nombre K de clusters pertinent est la valeur de K pour laquelle on observe un coude.

Ici : $K = 3$.

1 Introduction

- Apprentissage non supervisé et clustering
- Distance entre individus
- Distance entre groupes

2 Classification Ascendante Hiérarchique

- Principe général
- Exemple d'implémentation
- Utilisation de `scipy`

3 K-means

- Principe général
- Exemple

4 Évaluation des solutions obtenues

- WCSS et méthode du coude
- Propriété de la distance de Ward

Propriété de la distance de Ward

Voici une autre propriété qui justifie que l'on s'intéresse à la distance de Ward :

Propriété (admise)

Étant donné un ensemble de points que l'on souhaite partager en deux sous-ensembles A et B , la partition qui **minimise la somme des carrés au sein des clusters** est celle qui **maximise la distance de Ward (pour la distance euclidienne)**.

En effet, si G_A , G_B et G sont les centres de gravité respectifs des points de A , B et $A \cup B$, alors on peut montrer que la distance de Ward entre les deux ensembles A et B (pour la distance euclidienne) vérifie :

$$D_W(A, B)^2 = \sum_{X \in A \cup B} d_2(X, G)^2 - \sum_{X \in A} d_2(X, G_A)^2 - \sum_{X \in B} d_2(X, G_B)^2$$

(On l'admet ici)

Propriété de la distance de Ward

La somme $\sum_{X \in A \cup B} d_2(X, G)^2$ est constante pour l'ensemble $A \cup B$.

Le découpage en deux ensembles A et B qui maximise la somme

$$D_W(A, B)^2 = \frac{n_A \cdot n_B}{n_A + n_B} d_2(G_A, G_B)^2$$

est celui qui minimise la somme $\sum_{X \in A} d_2(X, G_A)^2 + \sum_{X \in B} d_2(X, G_B)^2$,
c'est-à-dire la somme des carrés au sein des clusters

Autrement dit, éloigner le plus les deux centroïdes G_A et G_B (avec le facteur $\frac{n_A \cdot n_B}{n_A + n_B}$) revient à rapprocher le plus les points de chaque groupe de leur centre de gravité.